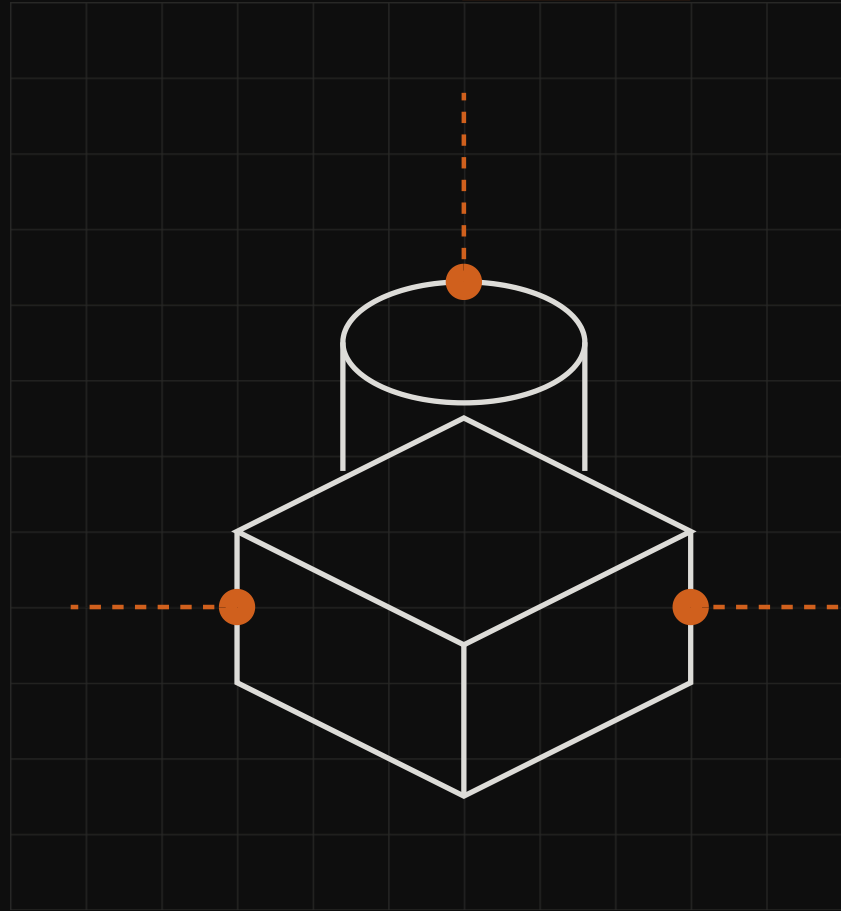




LEAD MAGNET KIT · BIM / REVIT / DYNAMO

Automatiza tus familias de Revit.

Un método corto para dejar de repetir clics: detecta lo que se repite, conviértelo en datos y deja que Dynamo haga el trabajo. Revisa todo antes de que llegue al proyecto.

01

Guía

Cómo detectar las tareas repetitivas al crear familias.

02

Plantilla Dynamo

Crea y asigna parámetros desde una tabla CSV. Adjunta a este PDF.

03

Checklist

Parámetros, geometría y funcionamiento antes de cargar.

ANTES DE EMPEZAR

Una familia bien hecha no debería hacerse dos veces.

Crear familias de equipos suele ser más copia y pega que diseño: los mismos parámetros, los mismos materiales, los mismos datos de fabricante, conectores que hay que dimensionar uno por uno. Este kit te ayuda a separar lo que **sí** requiere tu criterio de lo que puede hacer una máquina.

El método en 4 pasos

01

Detectar

Lista lo que repites y mide cuánto te cuesta.

02

Estandarizar

Un solo nombre, un solo tipo de dato, un solo grupo por parámetro.

03

Automatizar

Pasa el estándar a una tabla y deja que Dynamo la aplique.

04

Verificar

Revisa con checklist antes de cargar al proyecto.

Qué incluye

01 · PDF

Guía de detección

Señales de repetición, matriz para priorizar y hoja de trabajo para tu propio inventario.

02 · Adjunta al PDF

Plantilla Dynamo

Gráfico .dyn, código y CSV adjuntos a este PDF; código completo en el Anexo.

03 · PDF

Checklist de control

41 puntos en 6 bloques para revisar la familia antes de entregarla.

REQUISITOS

Revit 2022 o superior con Dynamo 2.x (motor CPython3). No necesitas paquetes externos: la plantilla usa solo nodos nativos y un nodo Python.

PARTE 01

Detecta lo que se repite.

Si haces lo mismo por tercera vez, ya es un proceso. A la tercera vez, escríbelo; a la cuarta, automatízalo.

8 señales de que una tarea debe automatizarse

- **Copias parámetros a mano** de una familia a otra, o los vuelves a crear desde cero.
- **El mismo dato tiene tres nombres:** Largo, Longitud, L. Las tablas no cuadran.
- **Llenas datos de fabricante y de información** (modelo, peso, fuente, nivel de desarrollo) tipo por tipo.
- **Creas materiales y subcategorías** uno a uno, y nunca quedan iguales entre familias.
- **Colocas conectores** y luego ajustas a mano su tamaño, sistema y dirección de flujo.
- **Tecleas los tipos de un catálogo** que ya existe en una tabla del fabricante.
- **Revisas "a ojo"** antes de cargar, y los errores aparecen ya en el proyecto.
- **Rehaces la familia completa** cuando el proveedor cambia un modelo o una medida.

Separa datos de lógica

El cambio de mentalidad más útil: una familia es **datos** (medidas, nombres, valores, conexiones) más **lógica** (cómo se construye). Los datos cambian en cada equipo; la lógica casi nunca.

DATOS · un archivo por familia

- Tabla de parámetros: nombre, tipo, grupo, valor.
- Medidas tomadas de la ficha técnica o del plano del proveedor.
- Conexiones: servicio, tamaño, posición, sentido de flujo.

LÓGICA · una herramienta para todas

- Crear lo que falta, actualizar lo que existe.
- Convertir unidades y validar valores.
- Dejar un registro de qué se hizo y qué falló.

Cuando cambie el modelo del proveedor, editas la tabla y vuelves a correr la herramienta. No rehaces la familia.

Prioriza: frecuencia × tiempo × riesgo

Califica cada tarea de 1 a 3 en los tres criterios y súmalos. **7–9** automatiza ya, **5–6** estandariza primero, **3–4** déjala manual.

CRITERIO	1 · BAJO	2 · MEDIO	3 · ALTO
Frecuencia	Una vez por proyecto	En cada familia	En cada tipo o cada actualización
Tiempo	< 5 min	5–30 min	> 30 min
Riesgo de error	Se nota enseguida	Se nota al etiquetar o tabular	Se nota en obra o en coordinación

Ejemplos típicos

TAREA	F	T	R	TOTAL	DECISIÓN
Crear el set de parámetros de información (LOI) en cada familia	3	2	3	8	Plantilla Dynamo + CSV
Asignar fabricante, modelo y peso a cada tipo	3	2	2	7	Columna Valor del CSV
Dimensionar conectores y ligarlos a parámetros	2	2	3	7	Script por familia
Crear materiales y subcategorías estándar	2	2	2	6	Plantilla de familia
Modelar la geometría singular de un equipo	1	3	2	6	Manual, con checklist

Errores comunes al automatizar

- **Ejecutar en modo Automático.** Usa modo Manual y un interruptor EJECUTAR: cada cambio de ruta no debe tocar la familia.
- **No simular.** Corre primero en modo revisión y lee el registro antes de aplicar.
- **Cambiar de ventana en Revit.** Dynamo trabaja sobre el documento desde el que se abrió; si cambias de archivo, se bloquea.
- **Olvidar las unidades.** Revit guarda longitudes en pies internamente: convierte siempre desde mm.
- **Mover archivos sueltos.** Mantén juntas la plantilla, las tablas y las librerías, con una estructura de carpetas fija.
- **Sin registro.** Si el script no dice qué creó y qué falló, no sabrás qué revisar.
- **Editar el archivo de compartidos a mano.** Un GUID duplicado rompe etiquetas y tablas en todo el proyecto.
- **Parametrizar todo.** Un equipo de catálogo no cambia de tamaño: parametriza lo que cambia (conexiones, datos, visibilidad) y deja fija la geometría del fabricante.

Tu inventario de tareas repetitivas.

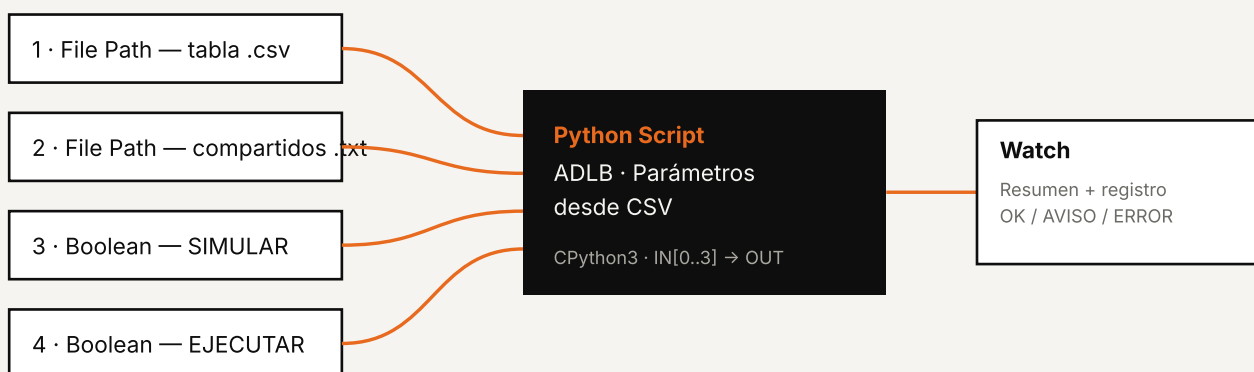
[illegible]

PARTE 02

Parámetros desde una tabla.

Escribes los parámetros una vez en un CSV (Excel sirve) y la plantilla los crea o actualiza en la familia abierta, con su tipo de dato, grupo, valor o fórmula, en todos los tipos a la vez.

El gráfico



Cómo usarla

1 · Prepara la tabla

Copia `ADLB_Parametros_ejemplo.csv`, editála en Excel y guárdala como *CSV UTF-8*. Una fila por parámetro.

2 · Abre la familia

Abre el `.rfa` en el Editor de familias y lanza Dynamo **desde ahí** (Gestionar > Dynamo). Abre `ADLB_Parametros_desde_CSV.dyn`.

3 · Simula

Selecciona el CSV, deja `SIMULAR = True`, pon `EJECUTAR = True` y pulsa Ejecutar. El registro muestra qué crearía y valida cada valor.

4 · Aplica

Si no hay errores, cambia `SIMULAR = False` y ejecuta de nuevo. Todo va en una transacción: **Ctrl+Z** la deshace completa.

DÓNDE ESTÁN LOS ARCHIVOS

Este PDF trae **adjuntos** el gráfico `.dyn`, el código `.py` y el CSV de ejemplo: ábrelos desde el panel de adjuntos (icono de clip) de tu lector de PDF. El código completo también está en el **Anexo**, listo para copiar y pegar en un nodo Python.

Columnas del CSV

COLUMNA	QUÉ PONER	VALORES VÁLIDOS
Nombre	Nombre exacto del parámetro, sin espacios.	Peso_kg, Conexion_Entrada_DN
TipoDato	Tipo de dato del parámetro.	Longitud · Texto · Numero · Entero · SiNo · Angulo · Area · Volumen · Material · URL
Grupo	Grupo de la ventana de propiedades.	Dimensiones · Restricciones · Datos · Identidad · Materiales · Texto · Graficos · Mecanico · Electrico · Fontaneria · Estructural · General
Instancia	Si = instancia, No = tipo.	Si · No
Compartido	Si = se toma del archivo de compartidos por nombre.	Si · No
Valor	Se asigna a todos los tipos. Longitudes en mm, ángulos en grados, material por nombre.	1200 · Si · Acero · EQ-01
Formula	Fórmula de Revit. Si hay fórmula, se ignora Valor.	Largo * Ancho

Ejemplo

```
# Las lineas que empiezan con # se ignoran. Separador ; o ,
Nombre;TipoDato;Grupo;Instancia;Compartido;Valor;Formula
Largo;Longitud;Dimensiones;No;No;1200;
Ancho;Longitud;Dimensiones;No;No;600;
Area_Huella;Area;Dimensiones;No;No;;Largo * Ancho
Holgura_Mantenimiento;Longitud;Restricciones;Si;No;800;
Codigo_Equipo;Texto;Identidad;Si;No;EQ-01;
Peso_kg;Numero;Datos;No;No;350;
Nivel_Desarrollo;Texto;Datos;No;No;350;
Conexion_Entrada_DN;Longitud;Mecanico;No;No;100;
Mostrar_Zona_Servicio;SiNo;Graficos;Si;No;Si;
```

Lo que verás en el registro

```
SIMULACION (no se modifiko nada): 16 filas | 16 nuevos | 0 existentes | 0 errores
CREAR fila 1 Largo (Longitud, tipo)
valor: 1200
CREAR fila 4 Area_Huella (Area, tipo)
formula: Largo * Ancho
AVISO fila 7 Peso_kg ya existe como instancia (el CSV dice tipo): no se cambia
ERROR fila 9 Material_Cuerpo -> material "Acero" no existe en la familia
```

LO QUE HACE

- Crea lo que falta y actualiza lo que existe.
- Asigna el valor a todos los tipos.
- Valida unidades y materiales antes de aplicar.

LO QUE NO HACE

- No borra parámetros.
- No cambia un parámetro de tipo a instancia: te avisa.
- No modifica tu archivo de compartidos.

PARTE 03

Antes de cargar al proyecto.

Marca cada punto. Si alguno no aplica, táchalo: una casilla vacía es una duda.

A Base de la familia

- ☐ Plantilla y **categoría** correctas (equipo mecánico, accesorio...).
- ☐ **Origen** en un punto útil para colocar y acotar (eje, anclaje, borde de base).
- ☐ Planos de referencia con **nombre** en ejes, anclajes y conexiones.
- ☐ Nombre de archivo y de tipo según tu **estándar de nomenclatura**.
- ☐ Fuente de la geometría identificada (ficha, plano del proveedor, medición).

B Parámetros

- ☐ Nombres del estándar, **sin duplicados** ni variantes.
- ☐ **Tipo vs. instancia** decidido a propósito, no por defecto.
- ☐ Lo que se etiqueta o tabula es **compartido**.
- ☐ Cada parámetro en su **grupo** correcto.
- ☐ **Fórmulas** sin errores ni referencias circulares.
- ☐ **Valores** cargados en todos los tipos (ninguno en 0 sin querer).
- ☐ Fabricante, modelo, descripción y comentarios de tipo completos.
- ☐ Set de información (LOI) completo: código, sistema, peso, nivel de desarrollo, fecha.
- ☐ Lo pendiente está marcado como **POR CONFIRMAR**, no inventado.

C Geometría

- ☐ Medidas generales **verificadas** contra la ficha (largo, ancho, alto).
- ☐ Nivel de detalle acorde al **LOD** pedido, ni más ni menos.
- ☐ **Subcategorías** asignadas (cuerpo, motores, estructura, conexiones).
- ☐ **Materiales** ligados a parámetros, no fijos en el sólido.
- ☐ Piezas menores ocultas en nivel de detalle **Bajo** (pernos, manijas).
- ☐ **Zonas de servicio** modeladas y con visibilidad controlable.
- ☐ Sin sólidos sobrantes, vacíos sueltos ni geometría importada innecesaria.
- ☐ Se lee bien en **planta, alzado y 3D**.

D Conectores

- ☐ Tipo correcto: tubería, ducto, eléctrico.
- ☐ **Clasificación de sistema** y dirección de flujo definidas.
- ☐ **Tamaño ligado a un parámetro**, no escrito en el conector.
- ☐ Sobre la **cara de la brida** y mirando hacia afuera.
- ☐ Datos eléctricos: tensión, polos, carga.

E Funcionamiento

- ☐ **Flexiona** cada tipo en el editor: cambia valores y vuelve a los originales.
- ☐ Prueba **valores extremos** (mínimo, máximo, cero) y revisa que nada se rompa.
- ☐ Cárgala en un **proyecto de prueba**, no en el proyecto real.
- ☐ La **etiqueta** lee los parámetros compartidos.
- ☐ Aparece completa en una **tabla de planificación**.
- ☐ Conecta un tramo de tubería o ducto a cada conector y revisa el sistema.
- ☐ Se corta bien en planta y en sección.

F Entrega

- ☐ **Purga** elementos no usados.
- ☐ Peso del archivo razonable para su uso.
- ☐ Fecha de modelado y versión actualizadas.
- ☐ Copia en la **biblioteca** con el nombre final.
- ☐ Registro del script guardado junto a la familia.
- ☐ Al recargar en el proyecto: **sobrescribir** y revisar instancias.
- ☐ Pendientes anotados para la siguiente versión.

RESULTADO

- | | |
|----------------|---|
| 41 / 41 | Lista para el proyecto. |
| 35-40 | Carga con pendientes anotados y comunícalos al equipo. |
| < 35 | Vuelve al editor. Lo que falla en la familia falla multiplicado en el modelo. |

BUENA PRÁCTICA

Usa parámetros **compartidos** para todo lo que vayas a etiquetar o tabular en el proyecto (código del equipo, sistema, peso). Los de familia sirven para la lógica interna (medidas, fórmulas, visibilidad).

Pasa la checklist a tu script

Varios puntos de esta lista se pueden revisar solos: nombres fuera del estándar, parámetros sin valor, conectores sin tamaño ligado, materiales fijos. Empieza marcando a mano y, cuando un punto falle seguido, conviértelo en una línea del registro.

ANEXO

Código de la plantilla.

Copia todo el bloque en un nodo **Python Script** (motor CPython3) con 4 entradas: CSV, compartidos, SIMULAR y EJECUTAR.

ADLB_Parametros_desde_CSV.py · 1 / 4

```
# -*- coding: utf-8 -*-
"""
ADLB · Plantilla base · Crear y asignar parametros de familia desde un CSV
Revit 2022+ · Dynamo 2.x · nodo Python (CPython3 o IronPython 2.7)

USO
1. Abra la FAMILIA (.rfa) en el Editor de familias de Revit.
2. Desde esa familia abra Dynamo y luego ADLB_Parametros_desde_CSV.dyn.
3. Entradas:
    IN[0] ruta del CSV de parametros (ver ADLB_Parametros_ejemplo.csv)
    IN[1] archivo de parametros compartidos (.txt). Vacio si no usa compartidos.
    IN[2] SIMULAR (True = solo revisa y reporta, no modifica la familia)
    IN[3] EJECUTAR (False mientras edita las entradas; True para correr)
4. Corra el grafico en modo Manual y lea el registro de la salida.

COLUMNAS DEL CSV (separador ; o , - codificacion UTF-8)
Nombre      nombre del parametro
TipoDato     Longitud | Texto | Numero | Entero | SiNo | Angulo | Area | Volumen | Material | URL
Grupo        Dimensiones | Restricciones | Datos | Identidad | Materiales | Texto | Graficos |
              Mecanico | Electrico | Fontaneria | Estructural | General
Instancia    Si | No
Compartido   Si | No (Si = se toma del archivo de parametros compartidos, por nombre)
Valor        valor para TODOS los tipos (longitudes en mm, angulos en grados, SiNo = Si/No,
              Material = nombre del material de la familia). Vacio = no asigna.
Formula      formula de Revit (opcional). Si hay formula, se ignora Valor.

QUE HACE
- Crea el parametro si no existe; si ya existe, solo actualiza valor / formula.
- Avisas si un parametro existente difiere del CSV (instancia/tipo) en lugar de cambiarlo.
- Asigna el valor a todos los tipos de la familia.
- No borra nada. Todo queda en una sola transaccion: Ctrl+Z la deshace completa.
"""
import io
import os
import clr

clr.AddReference('RevitAPI')
clr.AddReference('RevitServices')
from Autodesk.Revit.DB import (FilteredElementCollector, GroupTypeId, Material, SpecTypeId, UnitTypeId,
                               UnitUtils)
from RevitServices.Persistence import DocumentManager
from RevitServices.Transactions import TransactionManager

csv_path = IN[0]
shared_path = IN[1] if len(IN) > 1 else None
simular = bool(IN[2]) if len(IN) > 2 else True
ejecutar = bool(IN[3]) if len(IN) > 3 else False

TIPOS = { # TipoDato -> (ruta SpecTypeId, conversion del valor)
    'longitud': ('Length', 'mm'),
    'texto': ('String.Text', 'txt'),
    'url': ('String.Url', 'txt'),
    'numero': ('Number', 'num'),
    'entero': ('Int.Integer', 'int'),
    'sino': ('Boolean.YesNo', 'bool'),
    'angulo': ('Angle', 'deg'),
    'area': ('Area', 'm2'),
    'volumen': ('Volume', 'm3'),
```

ADLB_Parametros_desde_CSV.py · 2 / 4

```

        'material': ('Reference.Material', 'mat'),
    }
    GRUPOS = {
        'dimensiones': 'Geometry', 'restricciones': 'Constraints', 'datos': 'Data',
        'identidad': 'IdentityData', 'materiales': 'Materials', 'texto': 'Text',
        'graficos': 'Graphics', 'mecanico': 'Mechanical', 'electrico': 'Electrical',
        'fontaneria': 'Plumbing', 'estructural': 'Structural', 'general': 'General',
    }
    SI = ('si', 'sí', 's', 'yes', 'y', 'true', '1', 'x')

def norm(t):
    t = (u'%s' % (t or u'')).strip().lower()
    for a, b in ((u'á', u'a'), (u'é', u'e'), (u'í', u'i'), (u'ó', u'o'), (u'ú', u'u'), (u' ', u'')):
        t = t.replace(a, b)
    return t

def spec_id(path):
    obj = SpecTypeId
    for part in path.split('.'):
        obj = getattr(obj, part)
    return obj

def leer_csv(path):
    with io.open(path, 'r', encoding='utf-8-sig') as f:
        lineas = [l.rstrip('\r\n') for l in f if l.strip() and not l.lstrip().startswith('#')]
        sep = ';' if lineas[0].count(';') >= lineas[0].count(',') else ','
        cab = [norm(c) for c in lineas[0].split(sep)]
        filas = []
        for n, l in enumerate(lineas[1:], 2):
            celdas = [c.strip() for c in l.split(sep)]
            celdas += [u''] * (len(cab) - len(celdas))
            fila = dict(zip(cab, celdas))
            fila['_fila'] = n - 1
            filas.append(fila)
    return filas

def convertir(valor, modo, doc):
    v = (u'%s' % valor).strip()
    if modo == 'txt':
        return v
    if modo == 'bool':
        return 1 if norm(v) in SI else 0
    if modo == 'mat':
        for m in FilteredElementCollector(doc).OfClass(Material):
            if norm(m.Name) == norm(v):
                return m.Id
        raise Exception(u'material "%s" no existe en la familia' % v)
    num = float(v.replace(',', '.'))
    if modo == 'int':
        return int(round(num))
    if modo == 'num':
        return num
    unidad = {'mm': UnitTypeId.Millimeters, 'deg': UnitTypeId.Degrees,
              'm2': UnitTypeId.SquareMeters, 'm3': UnitTypeId.CubicMeters}[modo]
    return UnitUtils.ConvertToInternalUnits(num, unidad)

def definiciones_compartidas(app, path):
    if not path or not os.path.exists(path):
        return {}
    previo = app.SharedParametersFilename
    try:
        app.SharedParametersFilename = path
        archivo = app.OpenSharedParameterFile()
        return dict((d.Name, d) for g in archivo.Groups for d in g.Definitions)
    finally:
        if previo:
            app.SharedParametersFilename = previo

```

ADLB_Parametros_desde_CSV.py · 3 / 4

```

def procesar(doc, app, filas, shared_defs, simular, log):
    fm = doc.FamilyManager
    if fm.CurrentType is None:
        fm.NewType(u'Tipo 1')
        log.append(u'AVISO la familia no tenia tipos: se creo "Tipo 1"')
    tipos = list(fm.Types)
    creados = actualizados = errores = 0
    for f in filas:
        nombre = f.get('nombre', u'')
        ref = u'fila %d %s' % (f['_fila'], nombre)
        try:
            if not nombre:
                raise Exception(u'sin nombre')
            tipo = TIPOS.get(norm(f.get('tipodato')))
            if tipo is None:
                raise Exception(u'TipoDato "%s" no reconocido' % f.get('tipodato'))
            grupo = getattr(GroupTypeId, GRUPOS.get(norm(f.get('grupo')), 'Data'), GroupTypeId.Data)
            inst = norm(f.get('instancia')) in SI
            comp = norm(f.get('compartido')) in SI
            fp = fm.get_Parameter(nombre)
            if fp is None:
                if simular:
                    log.append(u'CREAR %s (%s, %s)' % (ref, f.get('tipodato'), u'instancia' if inst else u'tipo'))
                else:
                    if comp:
                        d = shared_defs.get(nombre)
                        if d is None:
                            raise Exception(u'no esta en el archivo de parametros compartidos')
                        fp = fm.AddParameter(d, grupo, inst)
                    else:
                        fp = fm.AddParameter(nombre, grupo, spec_id(tipo[0]), inst)
                    log.append(u'OK creado %s' % ref)
                    creados += 1
            else:
                if fp.IsInstance != inst:
                    log.append(u'AVISO %s ya existe como %s (el CSV dice %s): no se cambia' % (
                        ref, u'instancia' if fp.IsInstance else u'tipo', u'instancia' if inst else u'tipo'))
                    actualizados += 1
            formula = f.get('formula', u'')
            valor = f.get('valor', u'')
            if simular:
                if formula:
                    log.append(u' formula: %s' % formula)
                elif valor:
                    convertir(valor, tipo[1], doc) # valida el valor aunque no lo asigne
                    log.append(u' valor: %s' % valor)
                continue
            if formula:
                fm.SetFormula(fp, formula)
            elif valor:
                v = convertir(valor, tipo[1], doc)
                for t in tipos:
                    fm.CurrentType = t
                    fm.Set(fp, v)
        except Exception as ex:
            errores += 1
            log.append(u'ERROR %s -> %s' % (ref, (u'%s' % ex).replace(u'\n', u' | ')))
    return creados, actualizados, errores

if not ejecutar:
    OUT = [u'EJECUTAR = False. Revise las entradas y ponga EJECUTAR = True.']
else:
    doc = DocumentManager.Instance.CurrentDBDocument
    app = DocumentManager.Instance.CurrentUIApplication.Application
    log = []
    if not doc.IsFamilyDocument:
        OUT = [u'ERROR Abra Dynamo desde una FAMILIA (Editor de familias), no desde un proyecto.']
    elif not csv_path or not os.path.exists(u'%s' % csv_path):
        OUT = [u'ERROR No se encuentra el CSV: %s' % csv_path]
    else:
        filas = leer_csv(csv_path)
        defs = definiciones_compartidas(app, shared_path)
        if not simular:

```

ADLB_Parametros_desde_CSV.py · 4 / 4

```
        TransactionManager.Instance.EnsureInTransaction(doc)
    c, a, e = procesar(doc, app, filas, defs, simular, log)
    if not simular:
        TransactionManager.Instance.TransactionTaskDone()
    modo = u'SIMULACION (no se modifiko nada)' if simular else u'APLICADO'
    OUT = [u'%s: %d filas | %d nuevos | %d existentes | %d errores' % (modo, len(filas), c, a, e)] + log
```

ADLB_Parametros_ejemplo.csv

```
# ADLB · Ejemplo de tabla de parametros. Las líneas que empiezan con # se ignoran.  
Nombre;TipoDato;Grupo;Instancia;Compartido;Valor;Formula  
Largo;Longitud;Dimensiones;No;No;1200;  
Ancho;Longitud;Dimensiones;No;No;600;  
Alto;Longitud;Dimensiones;No;No;900;  
Area_Huella;Area;Dimensiones;No;No;;Largo * Ancho  
Holgura_Mantenimiento;Longitud;Restricciones;Si;No;800;  
Angulo_Descarga;Angulo;Restricciones;No;No;45;  
Codigo_Equipo;Texto;Identidad;Si;No;EQ-01;  
Peso_kg;Numero;Datos;No;No;350;  
Nivel_Desarrollo;Texto;Datos;No;No;350;  
Fuente_Geometria;Texto;Datos;No;No;Ficha tecnica del fabricante;  
Confianza_Dimensional;Texto;Datos;No;No;Media;  
Fecha_Modelado;Texto;Datos;No;No;2026-01-15;  
Conexion_Entrada_DN;Longitud;Mecanico;No;No;100;  
Conexion_Salida_DN;Longitud;Mecanico;No;No;80;  
Mostrar_Zona_Servicio;SiNo;Graficos;Si;No;Si;  
Material_Cuerpo;Material;Materiales;No;No;;
```

PARA USARLO

Copia el texto en el Bloc de notas y guárdalo como .csv con codificación UTF-8, o ábrelo en Excel y usa *Guardar como* > CSV UTF-8.

EL SIGUIENTE NIVEL

De parámetros a familias completas.

Con el mismo principio (datos en un archivo, lógica en una herramienta) se puede generar la familia entera: geometría desde las cotas del fabricante, bridas y pernos, conectores con su tamaño ligado, zonas de servicio y todo el set de información. Un archivo de datos por equipo y una herramienta para todos.

01

Detecta

Usa la hoja de trabajo durante una semana.

02

Automatiza

Empieza por tu tabla de parámetros.

03

Comparte

Cuéntame qué tarea automatizaste primero.